

BOUSALA (بوصلة) — INVESTOR COMPASS

Master Instruction File for Development Agent

Last updated: June 2026 | Version: MVP 1.2

1. PROJECT IDENTITY

Name: Bousala (بوصلة) — Arabic for "compass"

Purpose: Personal investment portfolio accounting and intelligence platform

User: Single investor managing assets across Saudi, US, and global markets

Stage: MVP — functional, locally hosted, not yet deployed

Stack: Python 3 + Streamlit + JSON flat-file storage

Codebase: ~24,000 lines across 37 Python modules

2. GOVERNING PHILOSOPHY

ACCURACY BEFORE FEATURES.

DATA INTEGRITY BEFORE UI POLISH.

This is a portfolio accounting system. A wrong number is worse than a missing feature. Every change must preserve:

1. Correct P&L calculations

2. Consistent valuations across all tabs

3. FIFO cost basis integrity

4. FX conversion accuracy

5. Transaction history immutability

When in doubt, do NOT change the calculation engines.

When in doubt, add a test before adding a feature.

3. ARCHITECTURE — THE RULES

3.1 SINGLE SOURCE OF TRUTH

Transaction history (transactions.json) is the master record.

Holdings, allocation, P&L, weights, and risk are ALL derived.

Never store derived values as authoritative.

This file contains THREE transaction types:

BUY — share purchase (creates/increases a holding)

SELL — share sale (reduces/closes a holding, creates FIFO lots)

SETTLEMENT — non-trade cash event (dividend, fee, tax, zakat, etc.)

All three types live in the same file. One file, one truth.

3.2 CENTRALIZED VALUATION ENGINE

Every tab that shows portfolio totals MUST call:

`calculate_portfolio_valuation()`

from `portfolio/valuation.py`. No tab may independently

sum holdings or currencies. The valuation engine handles

FX conversion, weight calculation, and sanity checks.

3.3 LAYERED DATA ACCESS

UI code (`app.py`) NEVER calls external APIs directly.

All market data flows through dedicated provider layers:

- `market_data_router.py` → routes to correct provider

- `market_prices.py` → Yahoo Finance (`yfinance`)

- **sahmk_client.py** → Saudi market data (SAHMK API)
- **fx_rates.py** → FX rates (yfinance + defaults)
- **edgar/client.py** → SEC EDGAR API

3.4 SAFE ZONES — DO NOT TOUCH WITHOUT TESTS

These modules contain financial logic that must never be modified without running the full test suite before AND after:

| SAFE ZONE — PROTECTED MODULES |

| portfolio/valuation.py — Valuation engine |

| portfolio/holdings.py — Holdings + FIFO + Settlements |

| portfolio/closed_holdings.py — FIFO sell matching |

| portfolio/accounts.py — Account + cash tracking |

| portfolio/cash_ledger.py — Cash audit trail |

| fx_rates.py — FX cascade engine |

| market_data_router.py — Price routing layer |

UI changes must NEVER alter these modules' behavior.

If you need to change a safe zone module, you MUST:

- 1. State what you are changing and why**
- 2. Run dev_test_runner.py BEFORE the change**
- 3. Make the change**
- 4. Run dev_test_runner.py AFTER the change**
- 5. Confirm test count did not decrease**

3.5 ATOMIC FILE WRITES

All JSON persistence uses write-to-temp-then-rename

via `_safe_json_write()` to prevent data corruption.

Never use `open()+json.dump()` directly for data files.

3.6 ASSET IDENTIFICATION

Holdings are keyed by sequential asset IDs: `AST_NNNNNN` (e.g., `AST_000001`, `AST_000042`). Never use ticker symbols as dictionary keys — multiple holdings can share a ticker.

The counter lives in `portfolio/_asset_counter.json`.

3.7 TRANSACTION IDENTIFICATION

All transactions (BUY, SELL, SETTLEMENT) share one ID format:

`TXN_{uuid8}` (e.g., `TXN_3f7a2b1c`)

Generated via: `"TXN_" + str(uuid.uuid4())[:8]`

There is NO sequential counter. Do NOT create one.

3.8 SETTLEMENT ISOLATION

Settlements NEVER alter: quantity, avg_cost, cost_basis, FIFO lots, or realized P&L. They are a separate cash flow layer that affects total return and income/expense metrics.

The FIFO engine in `closed_holdings.py` processes only BUY and SELL. Settlements are invisible to it.

4. MODULE MAP

edgar_app/

|—— **app.py — Main Streamlit UI (all tabs, ~7000 lines)**

|—— **main.py — Entry point (imports app.py)**

|—— **dev_test_runner.py — Test suite (145+ tests)**

|

| — **portfolio/ — CORE ACCOUNTING ENGINE**

- | | — **holdings.py — Holdings CRUD, transactions, FIFO, settlements**
- | | — **valuation.py — Centralized valuation (the one source of truth)**
- | | — **accounts.py — Investment accounts + cash balance**
- | | — **closed_holdings.py — FIFO closed lots, realized P&L**
- | | — **cash_ledger.py — Cash movement audit trail**
- | | — **risk.py — Portfolio risk engine**
- | | — **state.py — Research watchlist state**
- | | — **delta.py — Delta detection (filing changes)**
- | | — **comparison_store.py — Filing comparison history**
- | | — **core_thesis.py — Investment thesis storage**
- | | — **thesis_importer.py — PDF/text thesis import**
- | | — **decision_ranking.py — Decision queue scoring**
- | | — **sahmk_discovery.py — Saudi market discovery engine**
- | | — **init.py — Package exports**

| — **ai/ — AI ANALYSIS LAYER**

- | | — **analyzer.py — Filing analysis (OpenAI)**
- | | — **market_intel.py — Market intelligence analysis**
- | | — **valuation.py — AI-driven valuation**
- | | — **explainability.py — Analysis explainability**
- | | — **evidence.py — Evidence scoring**
- | | — **comparator.py — Filing comparison**
- | | — **fetcher.py — Filing text fetcher**
- | | — **uploader.py — File upload + extraction**
- | | — **cache.py — Analysis cache + rate limiter**

- |
- | — **edgar/** — SEC EDGAR API
- | | — **client.py** — EDGAR API wrapper
- | — **filings.py** — Filing data models
- |
- | — **fx_rates.py** — FX rate engine (yfinance + defaults)
- | — **market_prices.py** — Yahoo Finance price fetcher
- | — **market_data_router.py** — Multi-provider price router
- | — **sahmk_client.py** — SAHMK (Saudi market) API client
- | — **ticker_validator.py** — Yahoo Finance ticker validation
- | — **asset_master.py** — Asset master lookups
- | — **command_center.py** — Command center tab
- | — **report_store.py** — PDF report generation + storage

5. DATA STORAGE

All data lives in portfolio/*.json (flat files, single-user).

File Contents Format

holdings.json Active holdings (keyed by AST_ID) dict

transactions.json BUY/SELL/SETTLEMENT log list (append-only)

closed_lots.json FIFO closed lots (realized P&L) list (append-only)

accounts.json Investment accounts + cash dict

cash_ledger.json Cash movement audit trail list

_asset_counter.json Next AST_NNNNNN sequence number dict

deleted_holdings.json Soft-deleted holdings archive list

portfolio_state.json Research watchlist entries dict

core_theses.json Investment thesis documents dict

delta_history.json Filing change alerts list

comparison_history.json Filing comparison records list

CRITICAL: These files live **INSIDE** the **edgar_app/portfolio/** directory.

On Replit's ephemeral filesystem, they are lost on redeploy.

This is a known limitation — migration to SQLite is planned.

6. UI TABS (app.py)

Tab Purpose

 **Holdings Portfolio positions table, Buy/Sell/Settlement/Edit/Delete**

 **Allocation Asset allocation charts (geography, sector, type, currency)**

 **Closed Holdings Realized P&L — completed trades with FIFO lots**

 **Accounts Investment accounts (brokerage, crypto, etc.)**

 **Transaction History Read-only audit log of all trades and settlements (with filters)**

 **Cash Ledger Full cash movement trail (deposits, withdrawals, trades, settlements)**

 **Decision Queue Ranked investment decisions by conviction score**

 **Portfolio Risk Risk analysis (concentration, correlation, drawdown)**

 **Command Center Portfolio dashboard overview**

 **Thesis Memory Stored investment theses**

 **Market Intel AI-driven market intelligence analysis**

 **Filing Search SEC EDGAR filing lookup and AI analysis**

 **Research Watchlist Tickers being researched (not yet owned)**

 **Upload Filing Upload and analyze custom documents (PDF/text)**

 **Test Runner Developer test suite (145+ tests)**

 **SAHMK Discovery Saudi market company discovery via SAHMK API**

IMPORTANT: All portfolio value displays use the shared valuation bundle computed once via `_load_valuation_bundle()` and passed to Holdings, Allocation, and other tabs. This ensures consistency.

7. TRANSACTION ENTRY POINTS (IMPORTANT)

All trade and settlement actions live on the  Holdings tab.

There are 7 entry points:

1. + Add New Position (top bar)

→ Opens dialog with dual mode:

Mode A: "Record Existing Holding" — import/transfer, no cash debit

Mode B: "Record New Buy Transaction" — full buy with cash debit

→ Has Yahoo Finance auto-validate, Saudi ticker suggestions, watchlist quick-fill, market inference, cash balance check

2. + Buy More (top bar)

→ Opens dialog with holding dropdown picker

→ Pre-fills price from existing holding

→ Cash check, correction mode, new avg cost preview

3. Sell / Close (top bar)

→ Opens dialog with holding dropdown picker

→ Full/partial close, P&L preview, FIFO tracking

4. Settlement (top bar)

- Opens dialog with holding picker or portfolio-level option
- Records dividends, fees, taxes, zakat, purification, adjustments
- Auto-creates cash ledger entry + updates account balance
- Does NOT change quantity, cost basis, or FIFO lots

5. Per-row Buy / Sell / Settle (appears when selecting a table row)

- Same dialogs but pre-targeted to selected holding

6. Bulk Upload

- CSV import for multiple positions at once

7. Per-row Edit / Del (appears when selecting a table row)

- Direct field correction / soft-delete with archival

The  Transaction History tab is READ-ONLY (audit log).

It does NOT have any recording forms. This is intentional.

All recording flows through the Holdings tab.

8. FX ENGINE — HOW CURRENCIES WORK

Base currency: User-selectable (default SAR).

All portfolio totals are expressed in the base currency.

FX resolution cascade (fx_rates.py → get_rate):

1. Same currency → rate = 1.0, source = "same"
2. yfinance direct → e.g., USDSAR=X, source = "live"
3. yfinance reverse → e.g., SARUSD=X → invert, source = "live"
4. USD pivot → from→USD + USD→base, source = "live"
5. Static defaults → hardcoded table, source = "default"

Never raises. Always returns a rate. Labels every rate with its source.

Cached in Streamlit session state with 5-minute TTL.

USD-pegged currencies (SAR, AED, QAR) have hardcoded fallback rates.

Non-pegged currencies (EUR, GBP) use approximate defaults that drift.

RULE: Values in different currencies are NEVER added directly.

Always convert to base currency first via the valuation engine.

9. FIFO ENGINE — HOW SELLS WORK

When a SELL is recorded:

- 1. `_build_fifo_queue()` replays ALL transaction history for that ticker**
- 2. Builds a queue of remaining open buy lots (oldest first)**
- 3. `execute_sell_fifo()` matches the sell quantity against the queue**
- 4. Creates ClosedLot records with per-lot realized P&L**
- 5. Fees are prorated proportionally across matched lots**
- 6. Holdings quantity is reduced; if zero, position shows as closed**

The FIFO queue is rebuilt from scratch on every sell — no stale state.

Holdings added via "Record Existing Holding" (no BUY transaction) use a synthetic fallback lot at `avg_cost` for FIFO matching.

CRITICAL: The FIFO engine only processes `side == "BUY"` and `side == "SELL"`.

SETTLEMENT transactions are invisible to it. This is by design.

A settlement NEVER creates, modifies, or closes a FIFO lot.

10. SETTLEMENT ENGINE — HOW ADJUSTMENTS WORK

10.1 WHAT IS A SETTLEMENT

A settlement is any financial event tied to an investment that is not a buy or a sell. Examples:

- Dividend received
- Brokerage fee discovered late
- Tax withholding
- Annual zakat payment
- Purification payment
- Miscellaneous correction

Settlements affect the investor's real return but do NOT change share quantity, cost basis, avg cost, or FIFO lots.

10.2 CATEGORIES

Category Direction Typical use

Dividend + inflow Cash dividends, fund distributions

Fee - outflow Brokerage fees, custody charges

Tax - outflow Withholding tax, capital gains tax

Zakat - outflow Annual zakat on investment holdings

Purification - outflow Haram income purification payment

Adjustment +/- either Corrections, refunds, miscellaneous

These are stored in SETTLEMENT_CATEGORIES in holdings.py.

10.3 SCOPE

Asset-level: Linked to a specific holding via `asset_id`.

Example: "Q2 dividend from Aramco (2222.SR)"

Portfolio-level: Not linked to any holding (`asset_id = ""`).

Example: "Annual zakat on total portfolio"

10.4 THREE-STEP STORAGE FLOW

When a settlement is saved, THREE writes happen:

Step 1 → Append Transaction (side="SETTLEMENT") to transactions.json

Step 2 → Append CashEntry to cash_ledger.json (if account linked)

Step 3 → Update Account.cash_balance (if account linked)

If no account is selected, Steps 2 and 3 are skipped.

The transaction record still exists for return calculations.

Cash Ledger type mapping:

Dividend → DIVIDEND

Fee → FEE

Tax → FEE

Zakat → FEE

Purification → FEE

Adjustment (+) → OTHER

Adjustment (-) → OTHER

10.5 TRANSACTION DATA MODEL

Settlements use the same Transaction dataclass as BUY/SELL, with three additional fields (all with defaults for backward compatibility):

settlement_amount: float = 0.0 # +income / -expense

settlement_category: str = "" # from SETTLEMENT_CATEGORIES

settlement_currency: str = "" # ISO currency code

For SETTLEMENT transactions:

side = "SETTLEMENT"

quantity = 0.0

price = 0.0

ticker = holding's ticker if asset-linked, "" if portfolio-level

asset_id = holding's AST_ID if asset-linked, "" if portfolio-level

10.6 FUNCTIONS

record_settlement() — Create a new settlement (3-step write)

edit_settlement() — Edit an existing one (reverse old + apply new)

delete_settlement() — Delete and reverse all cash effects

These are SEPARATE functions from **record_transaction()**.

record_transaction() is NOT modified for settlements.

10.7 WHAT SETTLEMENTS AFFECT

✓ **Total Return calculation (when implemented)**

✓ **Portfolio XIRR (when implemented)**

✓ **Per-holding return (when implemented, if asset-linked)**

✓ **Income / expense metrics (when implemented)**

✓ **Zakat / purification tracking (when implemented)**

✓ **Cash ledger audit trail (immediately)**

✓ **Account cash balance (immediately)**

✗ **Does NOT change share quantity**

✗ **Does NOT change avg cost or cost basis**

✗ **Does NOT change FIFO realized P&L**

✗ **Does NOT create or modify closed lots**

10.8 UI ENTRY POINTS

Top action bar (Holdings tab):

[+ Add New] [+ Buy More] [ Sell / Close] [ Settlement]

Settlement uses default button style (not primary).

Per-row action bar (when row selected):

[+ Buy] [ Sell] [ Settle] [ Edit] [ Del]

When triggered from per-row, the holding is pre-selected.

Transaction History tab:

Settlements appear alongside BUY/SELL in the history table.

A Type filter allows filtering: All / Trades / Settlements.

10.9 TESTS

Test group: SETTLE-01 through SETTLE-08 (minimum)

Covers: record, portfolio-level, FIFO isolation, no quantity change, validation (notes < 10 chars, amount = 0, invalid category), and cash ledger type mapping.

11. TEST GOVERNANCE

Test runner: dev_test_runner.py (145+ tests after Settlement feature)

Run from:  Test Runner tab in the app

Rules:

- Test count must NEVER decrease**
- All tests must pass before any release**
- Tests use synthetic sandbox data — never touch real portfolio files**
- Tests import the same engines the live app uses (no duplicated math)**

Test categories:

N01–N10 Portfolio accounting (holdings, transactions, cost basis)

A11 Valuation consistency (cross-tab, per-account, FX)

A10 Account deletion guards

DISC SAHMK Discovery integration

SDS SAHMK Discovery storage layer

BULK Bulk upload validation

ADD-POS Add-position workflows (dual-mode)

CLOSE Closed holdings persistence

ASSET-TYPE Asset type taxonomy

ALLOC-AT Allocation by asset type

SETTLE Settlement transactions (record, edit, delete, FIFO isolation, cash ledger mapping, validation)

Known: All tests should pass as of this version.

12. CHANGE LOG

12.1 June 2026 — Quality Review (v1.1)

An independent code review was performed:

FIXED: 8 failing tests (all were test-expectation bugs)

- **N09: Test used wrong dict key (ticker instead of AST_ID)**
- **ASSET-TYPE-02→06: Same key mismatch in _can_upsert()**
- **DISC-06: Checked for "ticker" param; function uses "asset_id"**
- **A11-07: Compared asset_ids against tickers (type mismatch)**

FIXED: Atomic writes for all JSON persistence

- **All save_*() functions now use write-to-temp-then-rename**
- **Prevents data corruption if process crashes mid-write**
- **Added _safe_json_write() helper to holdings.py and accounts.py**

FIXED: ASSET_RISK_CLASS mapping

- **Was missing 8 of 14 asset types (had stale keys like "Fund", "Gold")**
- **Now covers all 14: Stock, ETF, REIT, Mutual Fund, Sukuk, Bond, Cash, Precious Metal, Commodity, Real Estate, Private Equity, Private Asset, Crypto, Other**

FIXED: requirements.txt

- **Added missing: yfinance, pandas, fpdf2**

REFACTORED: Transaction entry points

- **REMOVED** the duplicate Buy/Sell form from the Transactions tab
- **ADDED** Quick Buy and Quick Sell dialogs to Holdings tab top bar
- **RENAMED** Transactions tab to Transaction History (read-only)
- **ENHANCED** Transaction History with filters and summary metrics

12.2 June 2026 — Settlement Feature (v1.2)

ADDED: Settlement transaction type (side="SETTLEMENT")

- **3 new fields** on Transaction dataclass (backward-compatible defaults)
- **SETTLEMENT_CATEGORIES:** Dividend, Fee, Tax, Zakat, Purification, Adjustment
- **record_settlement(), edit_settlement(), delete_settlement()** functions
- **3-step atomic storage:** transaction + cash entry + balance update
- **FIFO engine confirmed untouched** (settlements are invisible to it)

ADDED: Settlement UI

- **Settlement button** on Holdings tab top action bar
- **Settle button** on per-row action bar
- **Settlement dialog** with holding picker, category, amount, notes
- **Impact preview** showing what is and is NOT affected

ADDED: Transaction History enhancements

- **Type filter:** All / Trades / Settlements
- **Settlement rows display amount + currency + category**
- **Summary metrics include settlement count and net amount**

ADDED: 8+ new tests (SETTLE-01 through SETTLE-08)

13. KNOWN LIMITATIONS

Severity Issue

High JSON file storage — data lost on cloud redeploy

(ephemeral filesystem). Migrate to SQLite.

Medium No authentication — single-user only, no login

**Medium app.py is ~7000 lines — should be split into
per-tab modules (tabs/holdings.py, tabs/allocation.py, etc.)**

Medium Total Return / XIRR not yet calculated

(settlements are recorded but analytics not built yet)

Low PDF export has encoding issues with Arabic text

(needs arabic_reshaper + python-bidi)

Low FX default rates for EUR/GBP drift over time

(no staleness warning)

14. DEVELOPMENT RULES FOR THE AGENT

BEFORE ANY CHANGE:

- ☐ **Read this file**
- ☐ **Identify which modules are affected**
- ☐ **Check if any Safe Zone module is involved**
- ☐ **Run the test suite (Test Runner tab)**
- ☐ **Note the test count and pass rate**

DURING THE CHANGE:

- ☐ **Never modify safe zone modules without explicit user approval**
- ☐ **Never add direct API calls in app.py — use the provider layers**
- ☐ **Never store derived values as authoritative data**
- ☐ **Never use ticker symbols as dictionary keys for holdings**
- ☐ **Use `_safe_json_write()` for any new JSON persistence**

- ☐ **Keep the Transaction History tab read-only**
- ☐ **All trade and settlement recording flows through the Holdings tab**
- ☐ **Settlements NEVER modify quantity, avg_cost, or FIFO lots**
- ☐ **record_settlement() is separate from record_transaction()**
- ☐ **Transaction IDs use TXN_{uuid8} — no sequential counters**

AFTER THE CHANGE:

- ☐ **Run the test suite again**
- ☐ **Confirm test count did not decrease**
- ☐ **Confirm all previously-passing tests still pass**
- ☐ **If a new feature was added, add at least one test for it**

NEVER DO:

- ☒ **Delete tests**
- ☒ **Reduce test count**
- ☒ **Modify valuation.py without running full suite**
- ☒ **Add recording forms outside the Holdings tab**
- ☒ **Sum values in different currencies without FX conversion**
- ☒ **Use open()+json.dump() for data files (use _safe_json_write)**
- ☒ **Hardcode asset IDs or ticker-based dictionary keys**
- ☒ **Let settlements touch FIFO logic or cost basis**
- ☒ **Use a different transaction ID format for settlements**

15. NEXT PRIORITIES (for the agent)

Priority Task

1 Split app.py into per-tab modules

2 Migrate JSON storage to SQLite

3 Add basic authentication (password protection)

4 Total Return + XIRR engine (consumes BUY + SELL + SETTLEMENT)

5 Income / Expense analytics dashboard

6 Zakat obligation tracker

7 Purification tracker

8 Fix Arabic PDF encoding

9 FX staleness warning for non-pegged defaults

10 Data backup / export automation

11 Deployment packaging (Docker or Render config)

END OF MASTER INSTRUCTION FILE

Bousala Investor Compass — بوصلة المستثمر
